

# Grand Line Run

## Serverless Cloud Native Gaming Application

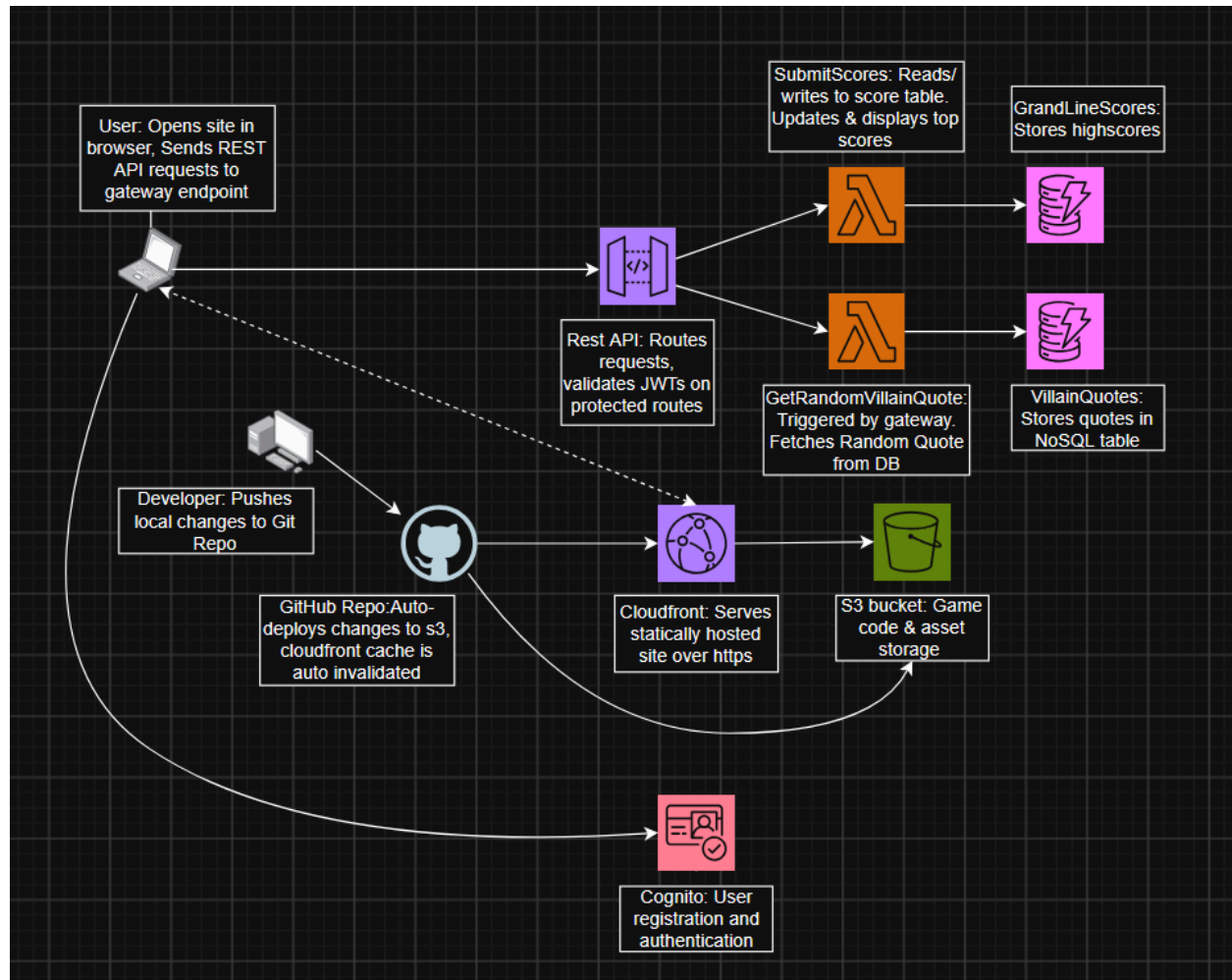
### Introduction

This project, Grand Line Run, is a multilevel flash game, based on the popular Japanese comic and Animated Series One Piece. The game consists of twelve distinct levels, each of which takes on the theme of a major story arc, progressing all the way from the first arc of the series to the arc that is currently being serialized.

The main characters in One Piece are a pirate crew who sail the seas all in pursuit of their own individual dreams. Each member of the crew receives particular attention in different arcs, and as such in each level you play as a different member of the crew or as a crew ally, with the goal of capturing a delicious cut of meat which is bouncing all around the screen. At each level, the villain of that story arc is also bouncing around and contact with that villain immediately prompts a game over screen. At each game over, a sinister quote is randomly selected from a set of that villain's in-series quotes and displayed to taunt the player, and the player is rewarded with something fitting the highest level that they cleared.

There are three possible quotes for each villain, at each death a random number between one and three is generated and the corresponding quote is fetched from a database to be displayed. The services I selected for Compute, Storage, Networking and Content Delivery, and Database respectively are listed in the following section.

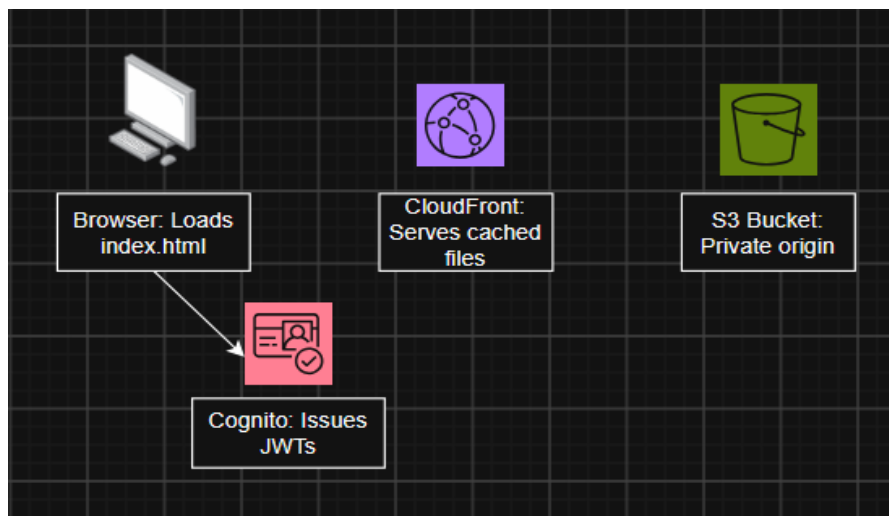
## Selected AWS Services and Application Architecture

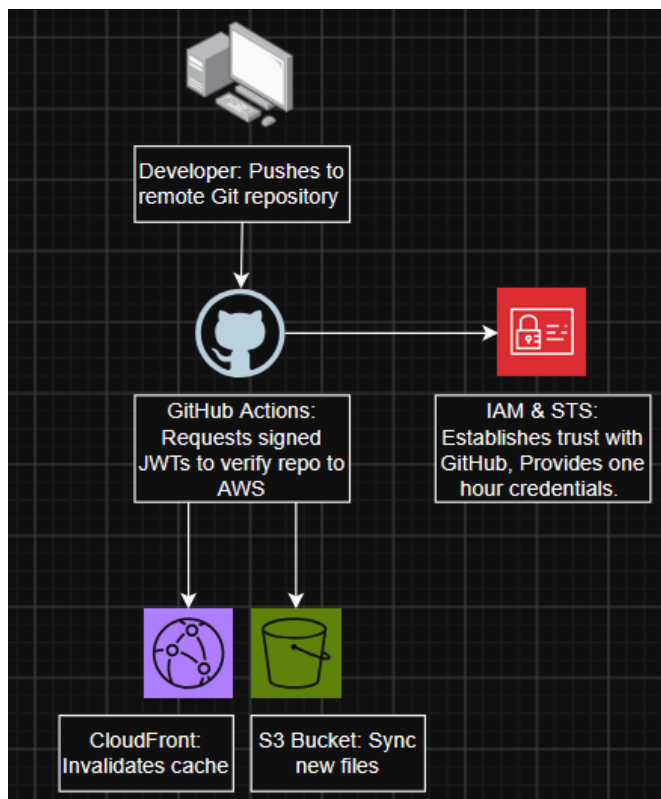
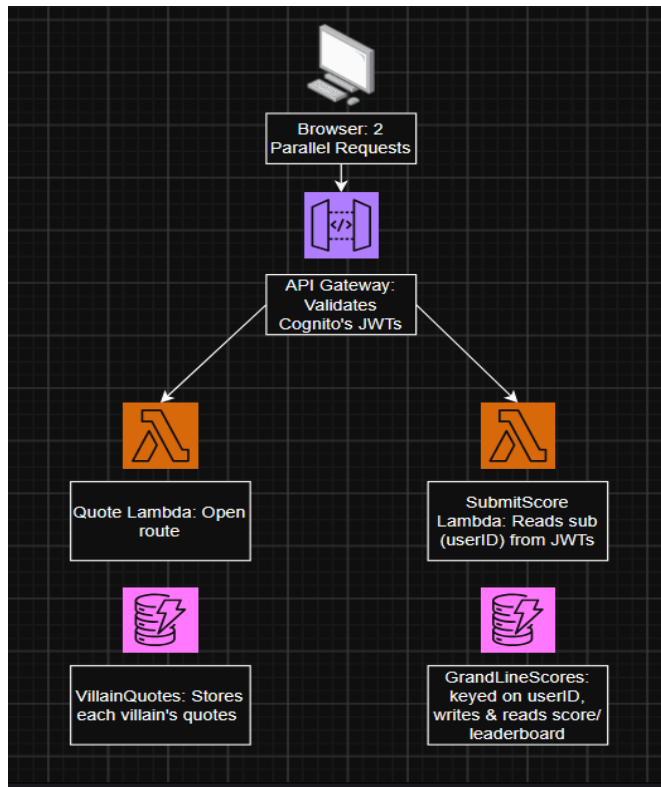


- Compute: AWS Lambda – Given the application’s event-driven design, lambda’s per request scaling is the best fit. Cost wise, lambda made the most sense for such a lightweight application since there are no idle costs.
  - getVillainQuotes: Used to access the *villainQuote* table, fetching quotes corresponding to each villain.
  - SubmitScore: GETs and displays leaderboard from *GrandLineScores* table, conditionally writing to the table upon a user’s highscore.
- Storage: Amazon Simple Storage Service (S3) – S3 was chosen primarily for its static web hosting, avoiding the operational overhead of services like EBS.
- Networking and Content Delivery:

- Amazon API Gateway – API Gateway was chosen for its routing of HTTP requests to lambda, and its hosting of the Cognito authorizer, validating JWTs before invoking any functions.
- Amazon Cloudfront – Selected for its redirection of http traffic to https, and its OAC which prevents access to the S3 bucket except through the cloudfront distribution. An added benefit is its higher availability, with the application's files being cached at several locations.
- Identity and Access Management:
  - IAM: Used for user creation, policy creation, and role creation and assignment.
    - The custom user CLI-Admin was provisioned for AWS Command Line Interface (CLI) use. This is more of a conceptual model of standard RBAC since the user does hold Administrator Access.
    - IAM OIDC established trust with GitHub actions. The custom role *GitHubActionsDeployRole* was created to grant permissions for remote code deployment to S3 and cloudfront cache invalidation, ensuring seamless development.
  - Identity as a Service (IDaaS): Amazon Cognito – Selected for its user authentication. User registration/login is required to access the game and have scores written to the leaderboard.
- Database: Amazon DynamoDB – This was the best fit for the project's serverless architecture. The NoSQL key-based retrieval was the best fit for my simple database since I did not require any of the relational capabilities of RDS for example.
  - Global Secondary Index (leaderboard): Secondary copy of the GrandLineScores data, sorting by score for the top 10 scores to be retrieved in a single query as opposed to a full table scan.

## Request Flow





**Definition:** *JWT (JSON Web Token): Used to verify that a valid session with an existing game user is taking place. The JWTs here are issued by Cognito, carrying verification of;*

- *The sender/issuer's identity (Cognito user pool): a url to the correct Cognito user pool, and a hash signed with the user pool's private key.*
- *The user's identity (Player): a unique immutable user ID (sub), the user's in-game username, and the user's email.*

## Application Data

- S3 (one-piece-game-mh245882):
  - Index.html (940.0B): Basic structure and layout of the UI.
  - assets/ (25.34MB): Contains the game's visual assets; 13 JPG files and 26 PNG files.
  - js/ (18.171KB): Contains four JavaScript files which handle the game's logic, user interaction, and communication with the backend API.
- DynamoDB
  - VillainQuotes:
    - Partition key (VillainID (N)): There are 12 unique enemies numbered 1-12, each with a corresponding VillainID.
    - Sort key (QuoteID (N)): Each villain has three unique quotes, numbered by QuoteID, 1-3.
      - Quote (String): String corresponding to each valid VillainID/QuoteID combination, storing that VillainID's QuoteIDth quote. For example (6,2) corresponds to the sixth unique enemy's second quote, the string "How unlucky for you to meet me..."
  - GrandLineScores:
    - Partition key (userID (String)): userID is a user's sub (unique immutable userID generated by Cognito for each user upon registration).
    - Global Secondary Index:
      - Partition key (leaderboard (String)): The leaderboard string represents the physical location in memory where the players' scores will be stored.
      - Sort key (highScore (number)): When highscores are stored here, they are scored in highscore descending, so with a single query the 10 user leaderboard can be retrieved, rather than a scan.
- Lambda (lambda\_function.py/GetRandomVillainQuote, 1.1KB): The lambda function is responsible for randomly selecting a quote from the current level's corresponding VillainID.

## Application's Deployment

- The presentation tier of my application consists of;
  - A CloudFront edge cache, obtaining the game's assets and code from the static S3 bucket, and serving them to the player's browser.
  - Cognito's managed login: Authenticates the user and issues JWTs for verification by the API-gateway and the game's js files.
- The application tier consists of the Restful API Gateway as an entry point that triggers the Lambda functions, GetRandomVillainQuote, and SubmitScores.
- The Data tier consists of two DynamoDB databases
  - VillainQuotes: Stores three quotes for each level's enemy to be selected via the lambda function.
  - GrandLineScores: See *Application Data*.

## Application Security

- The Presentation tier(S3): The application's assets are automatically protected via their bucket policy, which only allows reads from the application's unique CloudFront Distribution ID. Origin Access Control is configured on the S3 bucket's rest endpoint, and not a website endpoint, ensuring that data in transit between the bucket and the distribution is encrypted with HTTPS. CloudFront's DDoS protection and AWS WAF were enabled for this distribution.
- The Application tier (API Gateway and Lambda): The API endpoint is protected by default by AWS Shield Standard which absorbs large floods of incoming packets in the event of a DDoS attempt. However, it is susceptible to denial of wallet attacks via high-volume GET requests to either of the databases, since they are open to reads.
- Data Tier (DynamoDB): Currently access to the table is managed via IAM roles

## Programming Languages

The programming languages used were JavaScript, Python, and HTML. The entirety of the game's logic apart from the quote selection was written in JavaScript. I had initially programmed the game using Python's game development library Pygame, but since Pygame is not natively supported for internet play, I pivoted and did my best to translate all the game's logic to JavaScript. Python is used in the lambda function, which queries the DynamoDB table and fetches quotes. HTML was used to provide the basic UI structure. Code was required for the

backend API logic, the database querying, and the client side game mechanics which allowed the player to interact with the assets stored in S3.

## Cost Analysis

Up-front costs for a serverless architecture are \$0. To estimate the cost of the application I will estimate average monthly game attempts at 10000. This is much higher than the projected real number since it is not an application that has meaningful public advertisement or presence, but it gives a nice sizable number to work with and models standard live applications better than the realistic estimate.

- S3 (Costs \$0.023/GB) [1]:
  - Stored data = 27.1MB = 0.0271GB. This falls under S3's free tier of 5G. The monthly cost for storage comes out to  $0.0271\text{GB} \times \$0.023/\text{GB} = \$0.000623$ , this will not be included in the overall calculation since this is not subject to increase meaningfully as it does not scale.
  - Per-request scaling:
    - Index.html is never stored in any CloudFront cache, therefore, per attempt there is 1 GET request made to the S3 bucket for this specific file. 10000 base requests.
    - Assuming 25 CloudFront edge caches, there would be  $46 \times 25$  fetches, 1150 total. 1150 total GET requests =  $\sim \$0.0045$
- Lambda (Costs \$0.20/1Million Requests) [2]:
  - There are effectively 3 Lambda invocations per game attempt
  - The monthly cost comes out to  $30000/1000000 \times 0.20 = \$0.006$ .
- DynamoDB (Costs \$0.125 per million RRUs, \$0.625 per million WRUs)[3]:
  - WRUs (Write Request Units) and RRUs (Read Request Units) are the standard billing units for AWS DynamoDB.
  - Assuming an average of 10000 attempts monthly, I calculated approximately 12000 WRUs and 5000 RRUs
  - Summing WRUs and RRUs the monthly cost comes out to \$0.008125.
- API Gateway (Costs \$3.50 per million requests)[4]:
  - The number of API gateway requests is the same as that of the lambda: 3 requests per attempt, with 10000 attempts. The monthly cost comes out to  $30000/1000000 \times 3.50 = \$0.105$ .
- CloudFront: The CloudFront free tier covers 10 million requests to the distribution and 1TB of data transfer.
  - Given the 27.1MB of application data, it would take  $\sim 36901$  new users loading the game to exceed the free tier's maximum data transfer and see a cloudfront bill.

- The game's assets include 46 files. To be billed per request would need around 217392 new users.
  - If we were to calculate the data transfer and requests as in excess of the data transfer free tier, since this would be met sooner, the approximate cost for 10000 additional cold requests would be roughly \$23.00, a non-free tier calculation will be made excluding this, since it would require dramatically more users than what is already expected in the worst case scenario.
  - Cognito is free for the first 10000 MAUs (monthly active users). Given that the rest of my calculations assume 10000 total attempts, the number of active users would at most be 10000 assuming every new user plays only one game, falling into the free tier. Similar to CloudFront, Cognito will not be included in one of the non-free tier calculations. It would too dramatically affect the projected monthly cost, adding roughly \$150 which dwarfs all other tools combined.
1. Free tier: \$0.
  2. Free tier exceeded, excluding CloudFront & Cognito:  $\$0.0045 + \$0.006 + \$0.008125 + \$0.105 = \$0.12$
  3. Free tier exceeded, including CloudFront & Cognito: 173.12

## Future Evolution and Continued Development – Dec 18, 2025

First and foremost, to enhance the application's security following the Application Security section of this report, I would like to use Amazon CloudFront to encrypt data in transit, Amazon Cognito to authenticate users and requests, and I would assign a security role to the lambda function allowing only GET requests to the VillainQuotes table.

The original plan for this project was to implement a sign-up feature where a user's high score would be stored in the cloud and displayed alongside their username on the game's title screen for all players to see. This also would have been done via Amazon DynamoDB. As previously mentioned, signup and sign in would be implemented with Amazon Cognito and user credentials would be stored in a Cognito User Pool. I would also randomize aspects of the game such as player and villain images, such that at each level, you could pull a random player image and villain image, adding a unique flare to each attempt. This could also be done using DynamoDB, storing the unique image URLs as attributes in table entries.

From a development standpoint, I would like to find a way to host the original Python version of the game since the physics and controls were much smoother than the JavaScript counterpart. I will likely be going in and changing all the assets to images that are free of any



copyright as obviously I don't have any rights to the massive One Piece IP, but I did want to include this theme in the project and demo as I really love the series.

## References

- [1] “S3 pricing,” [aws.amazon.com/s3/pricing](https://aws.amazon.com/s3/pricing/), <https://aws.amazon.com/s3/pricing/> (accessed Dec. 18, 2025).
- [2] “Aws Lambda Pricing,” [aws.amazon.com/lambda/pricing](https://aws.amazon.com/lambda/pricing/), <https://aws.amazon.com/lambda/pricing/> (accessed Dec. 18, 2025).
- [3] “Amazon dynamodb pricing | nosql key-value database | Amazon Web Services,” [aws.amazon.com/dynamodb/pricing](https://aws.amazon.com/dynamodb/pricing/), <https://aws.amazon.com/dynamodb/pricing/> (accessed Dec. 18, 2025).
- [4] “Amazon API gateway pricing,” [aws.amazon.com/api-gateway/pricing](https://aws.amazon.com/api-gateway/pricing/), <https://aws.amazon.com/api-gateway/pricing/> (accessed Dec. 18, 2025).
- [5] S. Stormacq, “Amazon S3 encrypts new objects by Default | AWS News Blog,” [aws.amazon.com/blogs/aws/amazon-s3-encrypts-new-objects-by-default](https://aws.amazon.com/blogs/aws/amazon-s3-encrypts-new-objects-by-default/), <https://aws.amazon.com/blogs/aws/amazon-s3-encrypts-new-objects-by-default/> (accessed Dec. 18, 2025).
- [6] “Use HTTPS with CloudFront - amazon Cloudfront,” [docs.aws.amazon.com/AmazonCloudFront/latest/DeveloperGuide/using-https.html](https://docs.aws.amazon.com/AmazonCloudFront/latest/DeveloperGuide/using-https.html), <https://docs.aws.amazon.com/AmazonCloudFront/latest/DeveloperGuide/using-https.html> (accessed Dec. 18, 2025).
- [7] “Authentication service - customer iam (CIAM) - amazon cognito - AWS,” [aws.amazon.com/cognito/features](https://aws.amazon.com/cognito/features/), <https://aws.amazon.com/cognito/> (accessed Dec. 18, 2025).